

音声操作ロボットカー

プロジェクト概要と実行手順

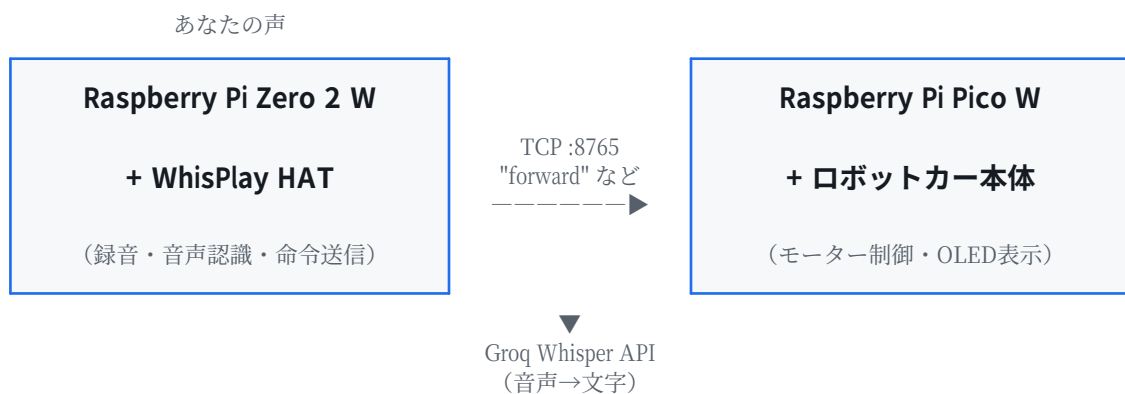
1. このプロジェクトについて

このプロジェクトは、声で操作できるロボットカーです。マイク・液晶画面・ボタンが付いた拡張ボード「WhisPlay HAT」を取り付けた Raspberry Pi Zero 2 W のボタンを押しながら「forward」「left」「right」「stop」のような命令を話すと、その音声は文字に変換され、WiFi経由で別の Raspberry Pi Pico W に送信されて車体が動きます。

音声認識には Groq 社の Whisper API（クラウド）を利用しています。無料枠の範囲内で利用でき、支払い方法を登録しない限り課金されません。

2. システム構成

本システムは、役割の異なる2台の装置で構成されています。



Raspberry Pi Zero 2 W + WhisPlay HAT

役割：ユーザーの音声を録音し、Groq のクラウド音声認識でテキスト化する。認識結果を "forward" などの命令に変換し、Pico W へ送信する。

Raspberry Pi Pico W + ロボットカー

役割：WiFi経由で受信した命令（forward / reverse / left / right / stop / speed:数値）に応じてモーターを制御する。OLED画面にバッテリー電圧とIPアドレスを表示する。

両者は同じWiFiネットワークに接続し、ポート 8765 の生のTCPソケット通信（HTTPやWebSocketは使わない、1行のテキストをやり取りするだけのシンプルな方式）でコマンドをやり取りします。

処理の流れ

- 1 ユーザーが WhisPlay HAT のボタンを押しながら話す
- 2 ボタンを離すと録音が終了し、音声ファイルが Groq の Whisper API へ送信される
- 3 認識されたテキストの中から、あらかじめ登録された単語（forward, left, right, stop など）を探す。まず完全一致を優先し、見つからない場合は条件付きで類似語も許容する

- 4 一致したコマンドを Pico へ TCP で送信する
- 5 forward / reverse は 2 秒間、left / right は 0.3 秒間（この車体でおよそ90度回転するように実測して調整済み）動作した後、自動的に停止する
- 6 画面と LED で状態（認識したテキスト、送信結果、エラーなど）を表示する

3. 必要なハードウェア

- Raspberry Pi Zero 2 W
- PiSugar WhisPlay HAT（液晶・マイク・スピーカー・RGB LED・ボタン付き拡張ボード）
- Raspberry Pi Pico W
- モーター4個を搭載したロボットカー車体
- SSD1306 OLEDディスプレイ（Pico側。バッテリー電圧とIPアドレスの表示に使用）
- 両方の装置が接続できるWiFi環境（本プロジェクトでは "spacecamp" というネットワーク名を使用）

4. セットアップ手順

4-1. Pico W 側の準備

- 1 MicroPython を Pico W に書き込む
- 2 モーター制御用ライブラリ（robot_car.py）と本体プログラム（11_WiFi_Control_test_3.py）を Pico へ転送する
- 3 プログラム内の SSID・パスワードを自分のWiFi環境に合わせて書き換える
- 4 Pico を起動すると自動的にWiFiへ接続し、OLED画面にIPアドレスが表示される。このIPアドレスは後でPi Zero側の設定に使うので控えておく
- 5 Pico はポート 8765 で待ち受け、コマンドを受信するとモーターを動かし、"OK" を返す

4-2. Raspberry Pi Zero 側の準備

- 1 WhisPlay HAT のオーディオドライバ（音声入出力用）と SPI（液晶画面用）を有効化する
- 2 必要なパッケージをインストールする

```
pip install groq websocket-client Pillow --break-system-packages
```

加えて sox（音声正規化用）と alsa-utils（マイク音量調整用）も導入する。

- 1 Groq の API キーを無料で取得する（console.groq.com/keys）。無料枠は回数制限のみで、支払い方法を登録しない限り課金されない
- 2 以下の環境変数を設定する

GROQ_API_KEY	Groq の API キー
PICO_HOST	Pico の IP アドレス（OLED に表示されたもの）
PICO_PORT	8765（Pico 側と一致させる）
MIC_GAIN	100（下の【重要】参照）
MOVE_DURATION_SEC	2.0（前進・後退の動作時間）

TURN_DURATION_SEC

0.3（左右旋回の動作時間。90度前後になるよう実測で調整済み）

△【重要】マイクゲイン（MIC_GAIN）について：初期設定の 80 のままだと録音音量が非常に小さく（実測でフルスケールの約5%程度）、認識精度が大きく低下することが実測でわかっています。ほぼ最大の 100 に設定することを強く推奨します。

4-3. 実行方法

- 1 Pico W の電源を入れ、WiFi接続が完了するまで待つ（OLEDにIPアドレスが表示される）
- 2 Raspberry Pi Zero 側で次のコマンドを実行する

```
python3 voice_car_control_groq.py
```

- 1 画面に "Voice Car Control (Groq)" と表示されたら準備完了
- 2 WhisPlay HAT のボタンを押しながら "forward" のように話し、ボタンを離す
- 3 画面に認識結果と送信結果が表示され、対応する動作を車体が行う

起動のたびに手動で実行するのが面倒な場合は、systemd サービスとして登録することで、Raspberry Pi の起動時に自動的にプログラムが立ち上がるようにできる。

5. 音声コマンド一覧

話す単語	送信されるコマンド	動作
forward / ahead / go	forward	前進（2秒間）
backward / reverse / back	reverse	後退（2秒間）
left	left	左折（約90度）
right	right	右折（約90度）
stop / halt	stop	停止

6. マイクを使わずに操作する方法

WhisPlay HAT のマイクを使わずに、SSH経由で直接コマンドを送ることもできる。

```
python3 send_command.py forward
python3 send_command.py right 0.5
python3 send_command.py stop
```

2行目のように末尾に数値を追加すると、その回転（動作）時間を一時的に上書きできる（90度になるよう調整する際などに使用する）。

これは、AIアシスタントに話しかけてロボットカーを操作する場合にも使っている仕組みで、マイクや音声認識を一切経由せず、コマンド文字列を直接 Pico へ送信する。

7. トラブルシューティング（開発中に実際に発生した問題と対策）

認識精度が低い場合

マイクゲイン（MIC_GAIN）を確認する。80のままだと録音音量が非常に小さく、認識精度に影響する。100に設定することを推奨する。

コマンド送信時に接続がタイムアウトする場合

Pico側の待受ポート番号（既定8765）と Pi Zero側の PICO_PORT の設定が一致しているか確認する。

WhisPlay HAT を長時間使用すると固まって再起動する場合

Raspberry Pi OS がデスクトップ環境（グラフィカルログイン）で起動していると、メモリが逼迫してシステムが不安定になることがある。CUI（ヘッドレス）起動に切り替えることで解決する場合がある。

```
sudo systemctl set-default multi-user.target
```

8. まとめ

本システムは、Raspberry Pi Zero 2 W（音声認識・HAT操作）と Raspberry Pi Pico W（車体制御）という役割分担で構成されている。通信には HTTP や WebSocket を使わない、単純なTCPソケット通信を採用しており、安定性と実装のシンプルさを重視している。曲がる角度や動作時間はハードウェア（モーターの速度や車体の重さなど）によって変わるため、実機での調整が必要である。